Strata — Hướng dẫn Hub & Child

Dành cho người quản trị và biên soạn tri thức

strata-kb v1.18.0 · 2026-09-19

1. Tài liệu này dành cho ai

Bạn sở hữu tri thức, hoặc bạn tạo ra tri thức.

- **Hub** là nơi tri thức của tổ chức được lưu và được tìm kiếm. Nếu bạn vận hành hub, bạn vận hành dịch vụ mà tất cả những người khác đọc từ đó, và bạn là cổng cuối cùng trước khi bất cứ thứ gì trở nên tìm được.
- **Child** là repository biên soạn. Nếu bạn làm việc trong một repo child, bạn đưa tài liệu vào, cho tóm tắt, kiểm tra, rồi publish lên hub.

Hai vai trò dùng chung bộ lệnh và chung mô hình bốn lớp nên được trình bày cùng nhau. Chỗ nào chỉ áp dụng cho một vai trò sẽ được nói rõ.

Có hai loại repository khác cũng publish lên cùng hub và có tài liệu hướng dẫn riêng: repo `dev` publish tri thức về chính code của nó (`<repo>-code`, `<repo>-svc`), và repo `ba` publish plan của nó (`<repo>-plan`: các mission và ticket đã sẵn sàng). Là người vận hành hub, bạn đăng ký cả hai như một child bất kỳ (mục 9.3).

Bạn không cần đọc tài liệu kiến trúc trước. Nhưng bạn cần hiểu một quy tắc trước mọi thứ khác:

> **Không thứ gì bạn soạn ra tìm được, cho tới khi một pull request trên hub được merge.** Kho cục bộ của bạn là bàn nháp. Hub mới là thế giới đã publish.

---

2. Cài đặt

```
pip install "strata-kb[ingest]"
kb --help
```

Extra `ingest` mang theo pipeline xử lý PDF. Không có nó thì mọi lệnh khác vẫn chạy — hữu ích cho máy chỉ truy vấn hoặc chỉ publish.

Yêu cầu. Python 3.11 trở lên, và Git. Nếu URL hub của bạn có kèm credential (dạng `https://x-access-token:…@github.com/...`), bạn cần **Git 2.31 trở lên** trên mọi máy và mọi CI runner. Git cũ hơn sẽ âm thầm bỏ qua cơ chế giữ token ngoài config của bản clone, và mọi lệnh liên quan tới hub đều lỗi xác thực. Debian 11 và Ubuntu 20.04 đều thấp hơn mốc này.

Muốn dùng Docker? `kb docker-setup` lo phần đó; xem mục 9.

---

3. Tạo repository

```
mkdir my-kb && cd my-kb && git init
kb init            # tương tác: hỏi bạn chọn kind nào
kb init --kind hub # hoặc không tương tác
```

`kb init` dựng khung theo kind bạn chọn và ghi lại vào `.kb/config.yaml`. Chạy lại về sau sẽ làm mới skill, template và workflow trong khi vẫn giữ dữ liệu — chỉ truyền `--force` nếu bạn thực sự muốn reset cả file dữ liệu.

3.1 Cấu hình hub

```
# .kb/config.yaml trên hub
kind: hub
repo_id: ops              # id entry của chính hub, nếu hub cũng soạn nội dung
asset_store:
 mode: none               # hoặc: {mode: s3, bucket: my-kb-assets}
```

3.2 Cấu hình child

```
# .kb/config.yaml trên child
kind: child
repo_id: safety           # id entry của bạn dưới federation/
hub: https://github.com/acme/kb-hub.git
```

`repo_id` trở thành tiền tố trong mọi trích dẫn mà repository này sinh ra (`safety:handbook §4.2`), nên hãy chọn tên ngắn và bền. Đổi về sau sẽ làm entry cũ trên hub mồ côi.

> **Hạn chế để credential trong `config.yaml` ở mức có thể.** URL `hub:` kèm token vẫn chạy, nhưng trên máy trạm hãy ưu tiên ssh (`git@github.com:acme/kb-hub.git`) hoặc URL https công khai, và để dạng có credential cho CI.

---

4. Đưa một tài liệu vào

```
kb ingest sources/employee-handbook.pdf \
 --id hr-handbook \
 --tags hr,policy,benefits \
 --revision "2026 edition"
```

Cờ	Tác dụng	Lời khuyên
`--id`	Id vĩnh viễn của tài liệu	Ngắn, chữ thường, nối bằng gạch ngang. Nó xuất hiện trong mọi trích dẫn mãi mãi.
`--tags`	Nhãn dùng để lọc trước khi tìm	2–5 tag. Hãy nghĩ theo cách người ta thu hẹp tìm kiếm, không phải cách bạn sắp xếp hồ sơ.
`--revision`	Nhãn phiên bản	Luôn đặt. Nó xuất hiện trong mọi trích dẫn và là cách người đọc biết câu trả lời đến từ bản nào.
`--sections`	Chỉ xử lý một số chương	Dùng khi chạy thử lần đầu trên tài liệu dài.
`--no-bookmarks`	Tách theo mẫu tiêu đề thay vì outline của PDF	Chỉ dùng khi outline thiếu hoặc sai.
`--no-summarize`	Bỏ qua bước tóm tắt tự động	Khi bạn muốn xem kết quả tách trước khi tiêu tốn lời gọi model.

4.1 File nguồn để ở đâu

PDF nguồn thường có bản quyền hoặc mang tính bảo mật. Chúng nằm trong `sources/`, thư mục mà bước dựng khung đã cấu hình Git bỏ qua. **Đừng commit chúng.** Cơ sở tri thức mang phần văn bản đã trích xuất do bạn kiểm soát; bản gốc ở nguyên chỗ hợp pháp của nó.

4.2 Ingest tạo ra những gì

```
.kb/hr-handbook/
├── _manifest.yaml              L1 — mọi section, đều "pending"
├── ch4-benefits.md             L2 — bảng đã nằm sẵn, văn xuôi để trống
├── ch4-benefits.raw.md         L3 — toàn văn bản gốc
└── assets/<sha256>.webp        hình, mỗi hình lưu một lần
```

Lần chạy đầu trên bất kỳ máy nào sẽ tải mô hình bố cục trang (~500 MB) rồi cache lại, nên chậm một lần và nhanh về sau.

4.3 Hãy đọc báo cáo ingest

Báo cáo không phải để trang trí. Nó nêu tên từng quyết định của bước tách: tiêu đề bị hạ xuống thành văn bản thường, id phải tự sinh, tiêu đề đánh số đến sai thứ tự, id trùng bị đổi tên, bookmark không khớp được, và phân bố kích thước section.

Hãy lướt qua trước khi đi tiếp. Một tài liệu sinh ra nhiều id dự phòng hoặc nhiều tiêu đề sai thứ tự thường là tài liệu có outline cần thay bằng `--no-bookmarks` kèm mẫu tiêu đề.

4.4 Ingest lại

- **Cả tài liệu** (không có `--sections`): thay thế toàn bộ. Mọi file Markdown của tài liệu đó bị xoá và ghi lại — kể cả những bản tóm tắt bạn đã review.
- **Một chương** (`--sections 6`): chỉ file và mục manifest của chương đó được ghi lại. Phần còn lại, kể cả tóm tắt đã review và trạng thái của chúng, không bị đụng tới.
- Một giá trị `--sections` không khớp tiêu đề nào sẽ **xoá** chương đó. Báo cáo nói rõ điều này; hãy đọc thay vì đoán.

---

5. Tóm tắt

Đây là bước duy nhất có model tham gia.

`kb ingest` chạy nó tự động, gọi một LLM CLI chạy ngầm mà nó tìm thấy trên `PATH` (`claude`, rồi `copilot`). Bạn có thể chỉ định bằng `--llm`, hoặc bỏ qua bằng `--no-summarize`.

```
kb status                   # còn gì chưa làm
kb summarize hr-handbook    # chạy hoặc chạy lại
```

Nếu không có LLM CLI nào được cài, các section đơn giản là ở nguyên trạng thái `pending`. Hãy mở coding agent của bạn và chạy `/kb-summarize` — nó đọc `kb status`, phân các section đang chờ ra nhiều sub-agent chỉ đọc chạy song song, rồi gộp kết quả lại theo đúng những quy tắc đó.

5.1 Những quy tắc model bị ràng buộc

Quy tắc	Vì sao
Viết bằng chính ngôn ngữ của tài liệu gốc	Bản tóm tắt bằng ngôn ngữ khác không chia sẻ từ vựng với bản gốc, và tìm kiếm từ khoá sẽ không còn khớp
Không bao giờ diễn đạt lại mã, tên trường, con số, đơn vị hay tham chiếu chéo	Đây chính là thứ người ta tra cứu tài liệu *để tìm*
Không bao giờ đụng vào bảng	Bảng do máy sao chép và được kiểm tra tự động; model sửa bảng là lỗi build
Thà giữ nguyên câu chữ gốc còn hơn bịa	Bản tóm tắt trung thực mà chưa hay thì còn sửa được; bản sai mà tự tin thì không

5.2 Chạy lại

```
kb summarize hr-handbook --redo --section 4.12       # một section
kb summarize hr-handbook --redo --dry-run            # xem kế hoạch trước
kb summarize --redo --all                            # toàn kho
kb summarize hr-handbook --print-prompt --section 4.12
```

`--redo` bỏ qua các section đã được người review, trừ khi thêm `--include-reviewed --yes`. Những section có rất ít văn xuôi được sao nguyên văn mà không gọi model.

---

6. Kiểm tra kết quả

```
kb build
```

```
kb build: OK
```

```
[error] hr-handbook §4.12: L2 table does not match L3 table
```

`kb build` chạy ba phép kiểm tra:

1. **Không còn chỗ trống.** Không có dấu `TODO`, không có tóm tắt rỗng.
2. **Mọi bảng khớp với bản gốc**, theo cả hai chiều. Đây là phép kiểm tra duy nhất không thể bỏ qua.
3. **Quy tắc chất lượng** — độ dài tóm tắt so với bản gốc, không có câu chỉ chép lại chính bảng của nó, không có mã in hoa bịa ra, đủ độ trùng từ vựng với bản gốc, L1 trong 25 từ, L0 trong 30 từ. Mặc định là cảnh báo; `--strict` biến chúng thành lỗi.

Build cũng thất bại nếu bản gốc thay đổi sau khi tóm tắt đã viết, hoặc bản tóm tắt thay đổi sau khi đã được duyệt. Và nó không bao giờ ghi manifest khi đang báo lỗi, nên một lần build hỏng không để lại thứ gì cập nhật dở.

```
kb build --allow-pending   # kiểm tra phần đã xong trong khi phần còn lại đang làm
kb build --strict          # mức mà `kb approve` yêu cầu
```

6.1 Sửa lỗi lệch bảng

Mở file `.raw.md` (L3) của section đó, chép nguyên bảng, dán đè lên bảng trong file `.md` (L2). Đừng gõ lại, và đừng định dạng lại — phép so sánh chỉ tha thứ cho căn lề và khoảng trắng, không tha thứ gì khác.

---

7. Duyệt (không bắt buộc nhưng nên làm)

```
kb approve hr-handbook
kb approve hr-handbook --by "Jane Smith <jane@acme.com>"
```

`kb approve` đòi cây làm việc của tài liệu đó sạch và `kb build --strict` phải pass. Nó ghi lại ai duyệt section nào, lúc nào, kèm hash của bản tóm tắt mà người đó đã duyệt — nhờ vậy một lần sửa sau đó vào bản tóm tắt đã duyệt sẽ bị phát hiện chứ không được mặc định là vô hại.

Duyệt là hành động của con người và không bao giờ tự động. Pull request trên hub vẫn là cổng thực sự kiểm soát việc publish; `kb approve` là cách bạn đánh dấu chuyên gia đã ký duyệt ở mức từng section bên trong đó.

---

8. Review một thay đổi

Nếu bạn review chứ không soạn, đây là toàn bộ công việc của bạn. Không cần lệnh nào — chỉ đọc diff.

- [ ] **Đọc phần văn xuôi mới** — file `.md`, không phải `.raw.md`. Nó có khớp với hiểu biết của bạn về chủ đề không?
- [ ] **Đối chiếu với tài liệu gốc.** Có bỏ sót gì quan trọng không? Có gì xuất hiện mà bản gốc không có không?
- [ ] **Kiểm tra từng mã, định danh, con số và đơn vị.** Phải đúng y như bản gốc.
- [ ] **Bảng** — bạn không cần soát từng dòng; build đã làm rồi và đã chặn PR nếu sai. Nhưng hãy liếc qua tìm lỗi *trích xuất*: một dòng hay một cột bị trình đọc PDF hiểu sai. Máy không bắt được lỗi đó.
- [ ] **Các dòng tóm tắt trong `_manifest.yaml`.** Chúng có nói đúng section nói về cái gì, đủ rõ để người tìm kiếm tìm ra không?
- [ ] Sai? Sửa thẳng file `.md` hoặc `.yaml` trên giao diện web, hoặc comment và hỏi người soạn.

Merge pull request này là chữ ký duyệt của chuyên gia. Nó vẫn chưa publish gì cả.

---

9. Publish

```
kb publish            # tự chọn chế độ phù hợp
kb publish --pr       # rõ ràng: mở hoặc cập nhật pull request trên hub
kb publish --direct   # rõ ràng: commit thẳng lên hub
```

Publish sao chiếu toàn bộ kho của bạn (cả bốn lớp) vào `federation/<repo-id>/` trên hub và dựng lại chỉ mục tổng hợp của hub.

Chế độ	Khi nào dùng
`--pr`	Thông thường. Cần git remote và `gh` mở được PR trên host đó.
`--direct`	Hub thuần cục bộ — một thư mục trên đĩa, không có remote. Bị từ chối trên hub được quản trị có remote.
Không cờ nào	Chỉ direct với hub không remote; còn lại là PR, hoặc từ chối rõ ràng.

Chỉ artefact được publish. Tài liệu, chỉ mục, manifest và tài nguyên ảnh thì đi; file cấu hình và dotfile ở lại và được nêu tên trong cảnh báo. Điều này là có chủ đích: trường `hub:` của bạn có thể chứa token, sao chiếu nó lên là publish luôn credential.

> Nếu `kb doctor` trên hub nêu tên những file nằm trong một entry mà một lượt publish không bao giờ ghi ra, đó là do phiên bản công cụ cũ để lại. Hãy publish lại để gỡ chúng đi, và **xoay vòng mọi bí mật chúng từng chứa** — xoá file không xoá được nó khỏi lịch sử Git.

9.1 Publish từ CI

File `kb-publish.yml` được dựng sẵn sẽ chạy `kb ci-publish` từ chính GitHub Actions job của bạn. Nó xác thực với dịch vụ intake của hub bằng OIDC của GitHub Actions, nên không có secret dài hạn nào phải lưu hay xoay vòng. Chính `kb publish` tạo và đẩy tag kích hoạt nó. Repo `dev` và `ba` publish theo cùng con đường này, từ `kb-code.yml` và `kb-publish.yml` của chúng.

9.2 Khi publish bị từ chối

Thông báo	Làm gì
`gh` không mở được pull request	Cài và xác thực `gh` cho host đó (`GH_HOST=<host>` với Enterprise) tới khi `gh repo view` chạy được bên trong bản clone hub
từ chối push trực tiếp lên hub được quản trị	Dùng `--pr`, hoặc `kb ci-publish` từ CI. Việc từ chối chính là cơ chế quản trị đang hoạt động.
`federation cycle detected`	Một chuỗi hub sẽ khiến nội dung quay vòng về chính nó. Kiểm tra `hub:` trên từng hub trong chuỗi.
mọi lệnh hub đều lỗi xác thực	Kiểm tra `git --version` — dưới 2.31 với URL hub có credential thì không gì chạy được
intake: repo "is not registered"	Repo chưa có trong `federation/registry.yaml` — xem mục 9.3

9.3 Đăng ký các bên publish trên hub (người vận hành hub)

Intake chỉ nhận publish từ một repository có trong `federation/registry.yaml` của hub, và chính mục đó — không bao giờ là payload — quyết định nó được ghi vào `repo_id` nào. Hãy thêm từng bên publish bằng một PR bình thường:

```
repos:
 acme/safety-docs: safety                # một child
 acme/payments: payments                 # một repo dev: -code và -svc
 acme/requirements:                      # một repo ba: -plan
   repo_id: acme-requirements
```

- `repo_id` phải bằng `repo_id:` trong `.kb/config.yaml` của repo đó. Nó cũng đặt tên cho các tài liệu được publish: repo `ba` có `repo_id: acme-requirements` publish ra `acme-requirements-plan`.
- Khoá registry là `owner/repo` của repo, ánh xạ tới `repo_id` của nó. Với publish kích hoạt bằng tag (mọi lần publish của `ba`, và repo `child` dùng `kb publish`), hãy để trống `workflow:`: intake so nó với toàn bộ `job_workflow_ref`, kết thúc bằng `@refs/tags/kb-publish/<timestamp>`, nên một ghim không bao giờ khớp và mọi lần publish sẽ bị từ chối. Lần push tag `kb-publish/*` khởi động `.github/workflows/kb-publish.yml`.

- Intake chỉ nhận `refs/tags/kb-publish/*`. Vì vậy repo `ba` publish khi BA chạy `kb publish`, không bao giờ tự động khi merge.
- BA cũng đặt `intake: <url>` trong `.kb/config.yaml` của repo (repo `ba` có sẵn thêm nó bằng tay rồi chạy lại `kb init --kind ba`).

Một tài liệu `<repo>-plan` đã publish xuất hiện trong federation giống như `-code`: trong index tổng hợp, với `kb query` và MCP (gắn tag `plan, generated`), và dưới menu **Plans** của web UI (mục 10.2). PR hub mà nó mở được review và merge như mọi PR khác. Nếu một PR đã đóng hoặc đã merge và nhánh `publish/<repo-id>` của nó đã bị xoá, lần publish kế tiếp mở một PR mới; bản sửa này đi cùng 1.17.0, nên hãy **triển khai lại dịch vụ intake** sau khi nâng cấp.

---

10. Vận hành dịch vụ hub

Một tiến trình phục vụ agent, chương trình và con người.

```
STRATA_KB_HTTP_TOKEN=<secret> \
 python -m strata_kb.mcp --hub . --transport http
```

Bề mặt	Dành cho	Xác thực
`http://host:8321/mcp`	AI agent	Bearer token
`http://host:8321/api/…`	script và tích hợp	Bearer token hoặc cookie
`http://host:8321/ui`	con người	đăng nhập bằng token

Hub là bắt buộc; server từ chối khởi động nếu không có, và chỉ tìm trong `federation/` — không bao giờ tìm trong `.kb/` làm việc của chính nó.

Tự pull. Server HTTP chạy `git pull --ff-only` trên checkout của hub mỗi 5 phút, nên một hub PR vừa merge sẽ tìm được ngay ở lần pull kế tiếp, không cần restart. Đổi chu kỳ bằng `STRATA_KB_HUB_PULL_INTERVAL` (giây; `0` = tắt). Checkout bị sửa tay hoặc lệch nhánh thì không bị đụng tới: pull thất bại, log cảnh báo, và server vẫn phục vụ bản đang có.

10.1 Docker

```
kb docker-setup       # hub: tạo .env, sinh token, khởi động dịch vụ
docker compose up -d
docker compose run --rm hub kb ingest source/handbook.pdf --id hr-handbook
```

Trên child, `kb docker-setup` kéo image ingest về để việc ingest không cần Python cục bộ.

10.2 Menu Plans

Web UI có menu **Plans** nằm cạnh Documents. Nó hiển thị những gì các repo `ba` publish dưới dạng tài liệu `<repo>-plan`, với trạng thái từng story được suy ra chỉ từ hub: **done** (id nằm trong một dòng `hist.*` của `-svc` của repo Dev), **ready** (ticket của nó đã publish), **not ready** (chưa có ticket đã publish) hoặc **blocked** (kèm lý do). Không có gì mới được lưu và không có đường ghi.

Route	Hiển thị
`/ui/plans`	Mọi repo BA và các mission của chúng, kèm số lượng theo trạng thái. `?repo=<id>` cho một repo
`/ui/plans/{mission}?repo=<id>`	Tóm tắt mission và bảng backlog
`/ui/plans/{mission}/{story}?repo=<id>`	Ticket. Ticket không có mission là `/ui/plans/tickets/{story}?repo=<id>`

Repo là tham số truy vấn vì id federation lồng nhau có chứa `/`. Không có `?repo=` thì dùng repo duy nhất đang giữ id đó; nếu có nhiều repo, trang là 404 nêu tên chúng. Tài liệu `*-plan` bị loại khỏi Documents, số đếm ở trang tổng quan và dải tag; một kết quả tìm kiếm nằm trong đó dẫn tới trang Plans của nó. Bản CLI tương ứng là `kb mission next --from-hub [--repo <ba-repo>]`.

10.3 Trước khi mở ra ngoài

- **Thay token được sinh tự động.** `kb docker-setup` sinh token cho tiện. Hãy coi đó là giá trị tạm.
- **Đặt số proxy tin cậy nếu đứng sau reverse proxy.** Mặc định là bỏ qua header chuyển tiếp. Để nguyên mặc định khi đứng sau proxy thì mọi người gọi đều trông giống proxy, và một người gọi có thể làm cạn giới hạn tần suất của tất cả.
- **Biết rõ giới hạn.** Xác thực chỉ là một bearer token — không OAuth, không SSO. Mức đó ổn cho mạng nội bộ và chưa sẵn sàng cho Internet công cộng.

Hướng dẫn triển khai đầy đủ: `docs/deploy-remote-mcp.md`.

---

11. Tìm kiếm

```
kb query "điều kiện nghỉ thai sản"
kb query "nghỉ thai sản" --tags hr --budget 600
kb get hr-handbook 4.12 --level l2
kb get hr-handbook 4.12 --level l3
kb tags
```

Cờ	Tác dụng
`--tags`	Chỉ tìm trong tài liệu mang các tag đó. Id tài liệu cũng dùng được như một tag.
`--budget`	Trần token. Chỉ mang tính khuyến nghị — kết quả đầu tiên luôn trả về trọn vẹn, và section không bao giờ bị cắt giữa chừng.
`--semantic`	Ép dùng nhánh ngữ nghĩa; cảnh báo nếu chưa cài embedding

Mọi kết quả đều trích dẫn `<repo-id>:<doc-id> §<section> (<revision>)`, nên bạn luôn biết repository nào, tài liệu nào, phiên bản nào đã trả lời.

Hãy truy vấn bằng chính ngôn ngữ của tài liệu. Kho không bao giờ dịch, nên truy vấn tiếng Anh trên tài liệu tiếng Việt sẽ khớp rất ít.

---

12. Giữ hub khoẻ mạnh

```
kb doctor      # kiểm tra cấu trúc; thoát khác 0 khi có vấn đề thật
kb reindex     # dựng lại federation/index.yaml khi nó lệch
kb stats       # kích thước token theo lớp, theo tài liệu
```

`kb doctor` báo cáo mục lục hỏng, sai lệch giữa manifest và file, nội dung bị sửa thẳng trên hub thay vì qua publish, file mà một lượt publish không bao giờ ghi ra, bản clone hub còn lưu credential trong Git config của chính nó, và Git cũ hơn mốc 2.31.

Hãy chạy nó sau khi nâng cấp và trước lượt publish đầu tiên sau đó. Một kho đang xanh có thể chuyển đỏ sau khi nâng cấp dù trên đĩa không có gì thay đổi — các phép kiểm tra chặt hơn, và thông báo sẽ cho bạn biết lệnh nào khắc phục.

---

13. Xử lý sự cố

Hiện tượng	Nguyên nhân	Cách xử lý
`kb build`: lệch bảng	Bảng bị sửa, thêm, nhân đôi hoặc đảo thứ tự trong bản tóm tắt	Chép nguyên bảng từ `.raw.md` dán đè lên bảng trong `.md`
`kb build`: còn section pending	Bước tóm tắt chưa xong hoặc bị bỏ qua	`kb status`, rồi `kb summarize`; hoặc `/kb-summarize` khi không có LLM CLI
`kb build`: "L3 changed after summarization"	Tài liệu bị ingest lại đè lên bản tóm tắt đang có	Chạy lại `kb summarize` cho các section bị ảnh hưởng
`kb ingest` lần đầu mất 10–30 phút	Mô hình bố cục đang tải về	Chờ một lần; các lần sau dùng cache
`kb: command not found`	Chưa kích hoạt virtualenv	`source .venv/bin/activate`
`kb query` không trả về gì	Không khớp tag hay nội dung, hoặc ngân sách quá nhỏ	Bỏ `--tags`, tăng `--budget`, và truy vấn bằng ngôn ngữ của tài liệu
Merge xong mà truy vấn vẫn không ra	Mới merge trên repo của bạn, chưa merge trên hub	`kb publish`, rồi merge PR trên hub
Mọi lệnh hub đều lỗi xác thực	Git cũ hơn 2.31 với URL hub có credential	Nâng cấp Git, hoặc dùng URL hub dạng ssh
MCP server không khởi động: không tìm thấy `python`	Cấu hình client gọi `python`, máy bạn chỉ có `python3`	Trỏ lệnh tới đường dẫn tuyệt đối của trình thông dịch

---

14. Tóm tắt lệnh

Lệnh	Mục đích
`kb init [--kind hub|child]`	Tạo hoặc làm mới repository
`kb docker-setup`	Chuẩn bị Docker: hub khởi động dịch vụ, child kéo image
`kb ingest <pdf> --id <id>`	Tách tài liệu, ghi L3, dựng khung L1/L2
`kb status`	Còn gì chưa làm
`kb summarize <doc>`	Điền hoặc điền lại tóm tắt
`kb build [--strict] [--allow-pending]`	Kiểm tra kho
`kb approve <doc>`	Ghi nhận chuyên gia ký duyệt từng section
`kb publish [--pr|--direct]`	Sao chiếu lên hub
`kb query <text>`	Tìm trên hub
`kb get <doc> <section>`	Lấy một section
`kb tags`	Liệt kê tag đã publish
`kb stats`	Kích thước token theo lớp
`kb doctor`	Kiểm tra sức khoẻ
`kb reindex`	Dựng lại chỉ mục tổng hợp

Mã thoát: `0` thành công, `1` lỗi hoặc cấu hình sai, `2` trích dẫn (hoặc cache hub) đã cũ.